RKE2 Kubernetes Cluster Setup

This document provides a brief overview of the single-node RKE2 Kubernetes cluster set up as a Proof of Concept (PoC).

- **Setup Overview**:
    - Cluster was setup using Ansible. The repository will be provided later.
    - RKE2 Kubernetes cluster, single-node, with Rancher Management UI installed
    - Installed apps: Docker registry, Postgres Cluster, Nginx Ingress
- **Accessing the Cluster**:
    - Use **kubectl** or **Rancher** management UI to interact with the cluster.
        - From the k8s server, `export KUBECONFIG=/etc/rancher/rke2/rke2.yaml` first and then you can execute kubectl commands
        - Rancher UI available at https://mgmtui.xulutions.net/
    - Port **6443** (api access, kubectl) is blocked by UFW firewall; whitelist your IP if needed.
        - ufw allow from ip address to any port 6443
- **Key Components**:
    - **Rancher**: Installed for cluster management.
    - **Cert-Manager**: Provides HTTPS certificates; certificates for the rancher and anonymizer pre-configured.
        - anonymizer.xulutions.net (default namespace)
        - mgmtui.xulutions.net

To create new certs you can import this yaml to k8s (kubectl apply -f yaml file. Or using rancher UI). Just adapt the name (can be anything), namespace (must be the same namespace where you deploy your app/ingress) and the dnsnames

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: example-com-tls
  namespace: my-namespace
spec:
  secretName: example-com-tls
  dnsNames:
  - example.com
  - www.example.com
```

```
issuerRef:
  name: letsencrypt-production
  kind: ClusterIssuer
usages:
- digital signature
- key encipherment
```

- **Docker Registry**: Installed for image storage
    - No ingress yet; pushing images from outside requires setting up an ingress or using kubectl proxy for a tunnel.
    - Tunnel example: kubectl port-forward svc/docker-registry -n registry 5000:5000 to access the registry locally from k8s-server via http://localhost:5000.
- **Postgres Cluster**: Managed by CloudNativePG for scalable PostgreSQL.
    - https://cloudnative-pg.io/
    - Connecting to postgres is possible only from within the cluster in the current config (can be changed later if needed)
        - psql –h endoregdblocal-rw.endopg.svc.cluster.local -p 5432 -d endoregDbLocal -U endoreg

- **Deploying Applications**:
    - Create **Deployments** under **Workloads** to start containers.
    - Configure a **Service** (type: ClusterIP) in the deployment to assign a DNS-routable address/IP within the cluster.
    - For external access, create an **Ingress**:
        - Set the domain as the host.
        - Point to the service.
        - Select HTTPS certificates. (must be created using cert-manager first)